Implementation of Additively Homomorphic and Quantum-Resistant Encryption on MapReduce

Aekkarin Yimyaem, Nattapol Phanmadee, Somchart Fugkaew, Uchupa Pao-in
Sirindhorn International Institute of Technology, Thammasat University, Thailand
6522772498@g.siit.tu.ac.th, 6522781994@g.siit.tu.ac.th, somchart@siit.tu.ac.th, 6522770666@g.siit.tu.ac.th

***Abstract*—The analysis of Adverse Event Reports (AER) plays a vital role in improving public health and medical safety. However, processing such sensitive data in cloud environments, such as MapReduce, raises significant concerns regarding privacy and post-quantum security. Over the years, many solutions have been proposed; NTRU encryption is one of them. NTRU encryption is a lightweight, additively homomorphic, and quantum-resistant scheme based on the difficulty of Shortest Vector Problem (SVP), making it suitable for secure computation over numerical data within the MapReduce framework. However, while NTRU enables privacy-preserving, its probabilistic encryption has some limitations. In particular, the probabilistic nature disrupts Shuffle-and-Sort incompatibility because the same plaintext does not map to the same ciphertext which affects the key-value pairs. To this end, we proposed a hybrid encryption by integrating Searchable Symmetric Encryption (SSE) for textual or categorical data to ensure identical plaintexts produce consistent ciphertexts, enabling reliable Sort-and-Shuffle operation. Additionally, we encrypt sensitive CSV files using Ciphertext-Policy Attribute-Based Encryption (CP-ABE) before uploading to the cloud to enforce fine-grained access control. To enhance MapReduce performance, we incorporate indexing mechanisms including Finite State Transducers (FSTs) for scalable keyword indexing and lookup. This integrated approach ensures secure, efficient, and scalable processing of encrypted AER data in untrusted cloud environments.**

Keywords— Medical Data, Adverse Event Report, MapReduce, Cloud Computing, Privacy-Aware, Homomorphic Encryption, Searchable Symmetric Encryption, NTRU, Quantum Resistant, Ciphertext-Policy Attribute-Based Encryption, Bloom Filters, Merkle Trees, Finite State Transducers

I. Introduction

Medical data is invaluable for analysis, as it facilitates the synthesis of new medical knowledge. One critical type of medical data is the Adverse Event Report (AER), which consists of reports detailing negative reactions experienced by patients in response to various medical treatments, including pharmaceutical drugs and medical devices. Analyzing AERs can provide valuable insights into drug safety, potential side effects, and patient risk factors.

One effective approach to handling and analyzing large volumes of such data is the MapReduce model of Apache Hadoop. The MapReduce framework is designed to efficiently process vast amounts of structured and unstructured data by distributing computational tasks across multiple nodes in a cluster. It achieves this by breaking down large datasets into smaller chunks, processing them in parallel, and aggregating the results. This parallelized approach significantly enhances scalability and performance, making it well-suited for big data applications, such as identifying patterns in adverse event reports, detecting correlations between medications and side effects, and improving post-market drug surveillance.

MapReduce's distributed processing capabilities make it particularly ideal for medical data, which is often vast, complex, and heterogeneous. Medical datasets like AERs include diverse formats, from structured fields to free-text notes, requiring flexible and scalable computational models. By leveraging MapReduce, healthcare analysts can efficiently handle high-volume, high-velocity data without compromising on performance or accuracy. The framework's fault tolerance and ability to scale horizontally ensure that sensitive medical computations can be executed reliably across multiple machines, even in the face of hardware failures. Moreover, MapReduce's batch-processing nature aligns well with periodic medical data analysis tasks such as drug safety monitoring, enabling timely insights that can lead to improved patient outcomes and more proactive healthcare interventions.

However, due to the sensitive nature of medical data, the need for encryption is paramount. Encryption in the MapReduce process is typically divided into two views: encryption for data at rest (in storage), and encryption for data in transit (communication). At present, the most commonly applied security measures in MapReduce include AES (Advanced Encryption Standard) for encrypting data at rest, Transport Layer Security (TLS) for securing communication between worker nodes, and Kerberos for authentication and authorization, offering robust security against unauthorized access. Its computational efficiency ensures fast encryption and decryption, which is critical for large-scale systems like Hadoop's HDFS.

With the advent of quantum computing, many traditional encryption schemes, such as RSA, are at risk of being broken. In response, research in post-quantum security has led to the development of several quantum-secure encryption methods. One of the most promising is N-th Degree Truncated Polynomial Ring Unit (NTRU) encryption, a quantum-secure public-key cryptosystem.

NTRU is based on polynomial rings and uses operations like addition, modular inversion, and convolutional products. It offers low memory usage and high speed compared to other cryptosystems. Its security relies on the hardness of the Shortest Vector Problem in lattices, and its key size is smaller than other numerical-based systems, improving performance. NTRU is unique in that it is independent of both discrete algorithmic problems (like those in ECC and ElGamal) and factorization (as in RSA), making it an ideal candidate for post-quantum cryptography. [5]

Additionally, NTRU encryption is a type of homomorphic encryption, a class of encryption schemes that

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

allows computations to be performed on ciphertext without decryption. Specifically, homomorphic encryption supports direct arithmetic operations—such as addition and multiplication—on encrypted data. Homomorphic encryption can be broadly categorized into two types: partially homomorphic encryption, which supports only one operation (either addition or multiplication), and fully homomorphic encryption, which supports both operations on ciphertexts. Efforts have been made to employ homomorphic encryption in the handling of medical data. [6] [7] [8]

Despite the advantages of NTRU encryption, there are several challenges that prevent its wholesale application onto a given dataset. The first problem is that the probabilistic nature of NTRU encryption means the same plaintexts are mapped to different ciphertexts, clashing with the operation of grouping key-value pairs based on the key part in MapReduce processes. Efforts have been made in the creation of schemes to allow searchable encryption without majorly compromising privacy. [9] proposes a similarity-based searchable encryption scheme using Locality-Sensitive Hashing (LSH) and NTRU encryption to enable privacy-preserving medical data sharing while ensuring quantum resistance. [10] proposes a searchable encrypted e-medical framework that enables multi-client access while ensuring both data confidentiality and efficient encrypted search. Their approach incorporates fine-grained access control and secure dynamic searchable encryption, balancing security with practical performance. To that end, we decide to employ the use of SSE with encrypted indexes, allowing the MapReduce process to group key-value pairs together without ever decrypting the data.

To strengthen data privacy and performance in encrypted data processing workflows, we introduce several enhancements to the MapReduce framework. First, we apply Ciphertext-Policy Attribute-Based Encryption (CP-ABE) to encrypt structured CSV files before uploading them to the cloud. This ensures fine-grained access control based on attribute-based policies, allowing only authorized users to decrypt specific parts of the data. Second, to support efficient multi-keyword search over encrypted text fields, we integrate indexing techniques including Bloom Filters for quick keyword filtering and Merkle Trees to provide verifiable search result integrity. Finally, we adopt Finite State Transducers (FSTs) through Apache Lucene to enable scalable and memory-efficient keyword indexing, allowing rapid lookup even in large-scale encrypted datasets.

In summary, with this paper, we intend to contribute by creating a proof of concept for the integration of quantum-secure encryption in MapReduce processes for medical data, particularly Adverse Event Reports data (AER), on top of implementing additive homomorphic encryption to compare with existing solutions.

II. Related Work

Previous studies have examined the use of homomorphic encryption for privacy-preserving computation and searchable symmetric encryption for secure word search. Nevertheless, their integration, especially in a MapReduce compatible way and quantum-resistant remains unexplored.

In [1], He et al. introduced a comprehensive view of searchable encryption schemes that incorporate homomorphic encryption, particularly emphasizing the use of additively homomorphic encryption such as Paillier because of its efficiency and simplicity. They categorize existing schemes by encryption type, search structure, and supported functionalities like multi-keyword and ranked search. However, the study notes limitations in current approaches, including minimal adoption of fully homomorphic encryption, lack of support for fuzzy search and verifiability, and limited exploration of quantum-resistant techniques.

In [2], Amorim and Costa conducted a focused analysis on the role of homomorphic encryption (HE) in searchable encryption (SE) systems, highlighting how HE enables computation over encrypted queries without compromising data confidentiality. Their review outlines the key characteristics of HE-based SE schemes, particularly those utilizing partially homomorphic encryption like Paillier. They emphasized the index-based search models and functionalities such as ranked retrieval and multi-keyword support. Nevertheless, the study also points out limitations, including the lack of fuzzy search support, limited user access control mechanisms, and insufficient post-quantum cryptographic resilience in current implementations.

In [3], Talapatra et al. proposed NOMOS, a dynamic multi-client SSE scheme enabling efficient conjunctive keyword searches with forward and backward privacy using token-based access and Redundant Bloom Filters to minimize leakage and support fine-grained user control. They also introduced a second in-progress schema which is NTRU-OQXT, a compact and quantum-safe SSE scheme based on NTRU lattices, achieving faster searches and reduced storage. Also, its extension, TWINSSE-NTRU-OQXT supports disjunctive queries using Fast-Fourier trapdoor sampling for efficient and secure search. However, quantum-safe schemes remain limited to static databases and support for dynamic updates.

In [4], Shinoki et al. presented fully secure SSE schemes using pseudorandom function, pairings, and lattice-based assumptions including NTRU. Their constructions achieve strong security guarantees, including trapdoor privacy, but are limited to static data and bounded queries. Also, there are challenges of lack of dynamic update support and practical evaluation for applying such schemes in scalable, distributed environments.

III. Preliminaries

A. NTRUEncrypt Process

NTRU is a lattice-based public key cryptosystem that supports both encryption and digital signatures through NTRUEncrypt and NTRUSign, respectively. It is resistant to quantum attacks, making it a strong post-quantum candidate.

NTRUEncrypt is based on the shortest vector problem in a lattice. It uses truncated polynomial rings : $R = Z[X] / (X^N - 1)$ where elements have the form : $a = a_0 + a_1X + a_2X^2 + ... + a_{N-1}X^{N-1}$ Each NTRU instance is defined by parameters (N,p,q). The system uses sets of polynomials Lf, Lg, Lm, Lr with small coefficients, usually from $\{-1,0,1\}$.

Key generation phase : Sending a secret message from Alice to Bob requires generating both public and private

keys. To generate random perturbations in the encryption process that can be successfully reduced in the decryption phase, Bob needs to select two small polynomials f and g, with f ∈ Lf, and computes the inverse f_p and f_q such that :

$$f \cdot f_p = 1 \pmod{p}, \quad f \cdot f_q = 1 \pmod{q}$$

If Bob finds in the process that the inverse of polynomial f does not exist in both rings at the same time, he needs to regenerate the polynomial f until the above condition is satisfied. After this process, Bob proceeds the next step of public key generation as follows :

$$h = p f_q \cdot g \pmod{q}$$

f, f_p and g are from Bob's private key and h is the final public key.

Encryption stage : When Alice wants to send a secret message to Bob, she needs to encode the message in the form of a polynomial m, where the polynomial needs to be in the interval [$-p/2, p/2$]. After creating the message polynomial, Alice randomly selects a polynomial with smaller coefficients r, which serves as a random perturbation. The randomly perturbed polynomials superimposed on the plaintext in the NTRU public-key cryptosystem are eventually reduced by addition and multiplication over the polynomial ring during the computation of the corresponding private key. Using Bob's public key h to compute the encrypted message e : $e = r \cdot h + m \pmod{q}$.

Decryption stage : According to the above encryption process, anyone who knows r can easily compute the message m, so r can not be revealed by Alice. In addition to the publicly available information, Bob knows his private key. The following computation is Bob's specific decryption process.

First, Bob multiplies the encrypted message e by his partial private key f

$$a = f \cdot e \pmod{q} = f \cdot (r \cdot h + m) = p \cdot r \cdot g + f \cdot m \pmod{q}$$

To ensure the correctness, the coefficients of the polynomial are chosen from [$-p/2, p/2$] for a. The following calculation is performed in the ring Rp :

$$b = a \pmod{p} = f \cdot m \pmod{p}$$

The key calculation is that the operation in the small modulus ring satisfies

$$pr \cdot g \pmod{p} = 0$$

To recover Alice's b and f_p multiplied by the message, Bob can use another part of his private key f_p , which can be computed as

$$c = f_p \cdot b = f_p \cdot f \cdot m \pmod{p} = m \pmod{p}$$

Since $f_p \cdot f = 1 \pmod{p}$, the decryption process is completed.

B. *Additive Homomorphic Encryption*

Homomorphic encryption allows computation over encrypted data without first decrypting it. Additive homomorphic encryption is one of the most efficient types particularly for privacy-preserving applications. In an additive homomorphic encryption scheme, ciphertexts can be added together to produce a new ciphertext which when decrypted yields the sum of the corresponding plaintexts. Mathematically, given two messages m_1 and m_2, and their respective encryptions $Enc(m_1)$ and $Enc(m_2)$, the scheme satisfies: $Enc(m_1) + Enc(m_2) = Enc(m_1 + m_2)$. This property enables computations such as summation of encrypted data streams or aggregation of values in a database while maintaining confidentiality.

Additive homomorphic encryption (AHE) enables the computation of sums directly on encrypted data, preserving confidentiality throughout the process. Lattice-based cryptographic schemes, such as NTRU and Learning With Errors (LWE), naturally support additive homomorphism due to their algebraic structures over polynomial rings or integer lattices.

C. *Searchable Symmetric Encryption*

Searchable Symmetric Encryption (SSE) is a cryptographic technique that enables keyword-based search over encrypted data without revealing the plaintext content. An index-based SSE scheme over a dictionary Δ is a collection of five polynomial-time algorithms:
SSE = (Gen, Enc, Trpdr, Search, Dec) such that:

$K \leftarrow \text{Gen}(1^k)$: A probabilistic key generation algorithm run by the user to set up the scheme.
It takes as input a security parameter k, and outputs a secret key K.

$(I, c) \leftarrow \text{Enc}(K, \mathbf{D})$: A probabilistic algorithm run by the user to encrypt the document collection.
It takes as input a secret key K and a document collection $\mathbf{D} = (D_1,\ldots,D_n)$ and outputs a secure index I and a sequence of ciphertexts $c = (c_1,\ldots,c_n)$.

$t \leftarrow \text{Trpdr}(K, w)$: A deterministic algorithm run by the user to generate a trapdoor for a given keyword.
It takes as input a secret key K and a keyword w, and outputs a trapdoor t.

$X \leftarrow$ Search(I, t) : A deterministic algorithm run by the server to search for the documents in **D** that contain a keyword w. It takes as input the encrypted index I and trapdoor t, and outputs a set X of (lexicographically ordered) document identifiers.

$D_i \leftarrow$ Dec(K, c_i) : A deterministic algorithm run by the client to recover a document. It takes as input a secret key K and a ciphertext c_i, and outputs a document D_i.

An index-based SSE scheme is correct if for all $k \in N$,
for all $K \leftarrow Gen(1^k)$,
for all $D \subseteq 2^{\Delta}$,
for all $(I, c) \leftarrow Enc(K, \mathbf{D})$,
for all $w \in \Delta$,

$Search(I, Trpdr_K(w)) = \mathbf{D}(w) \wedge Dec_K(c_i) = D_i$, for $1 \leq i \leq n$

D. Ciphertext-Policy Attribute-Based Encryption

CP-ABE is a type of ABE cryptographic scheme that incorporates the access policy into the ciphertext. This access policy specifies the conditions under which the encrypted data could be decrypted. Each user and data element is associated with a set of attributes such as user roles. The access policy is formulated using a combination of attributes, and decryption is possible only if the user's attributes match the policy in the ciphertext.

A ciphertext-policy attribute based encryption scheme consists of four fundamental algorithms: Setup, Encrypt, KeyGen, and Decrypt.

Setup. The setup algorithm takes no input other than the implicit security parameter. It outputs the public parameters PK and a master key MK.

Encrypt(PK, M, A**).** The encryption algorithm takes as input the public parameters PK, a message M, and an access structure A over the universe of attributes. The algorithm will encrypt M and produce a ciphertext CT such that only a user that possesses a set of attributes that satisfies the access structure will be able to decrypt the message. We will assume that the ciphertext implicitly contains A.

Key Generation(MK, S**).** The key generation algorithm takes as input the master key MK and a set of attributes S that describe the key. It outputs a private key SK.

Decrypt(PK, CT, SK). The decryption algorithm takes as input the public parameters PK, a ciphertext CT, which contains an access policy A, and a private key SK, which is a private key for a set S of attributes. If the set S of attributes satisfies the access structure A then the algorithm will decrypt the ciphertext and return a message M.

E. Elliptic Curve Cryptography

Elliptic Curve Cryptography (ECC) is a widely used public-key encryption method that offers strong security with relatively small key sizes, making it efficient for both computation and storage. ECC is based on the mathematical properties of elliptic curves over finite fields and is commonly used for secure communications, digital signatures, and key exchange protocols. Its security relies on the hardness of the elliptic curve discrete logarithm problem, which is difficult for classical computers to solve. However, ECC is vulnerable to quantum attacks, as quantum algorithms like Shor's could efficiently break its underlying assumptions. This limitation has driven the search for quantum-resistant alternatives, such as NTRU, especially for privacy-preserving applications in cloud-based frameworks like MapReduce.

An elliptic curve is a plane curve over a finite field which consists of the points satisfying the equation :

$$y^2 = x^3 + ax + b$$

Point Addition : Point addition refers to the operation of combining two distinct points P and Q on the elliptic curve to obtain a third point $R = P + Q$. This operation underpins the group structure of elliptic curves and is fundamental to all ECC-based computations.

Scalar Multiplication : Scalar multiplication involves adding a point P to itself repeatedly, defined as $kP = P + P + ... + P$ (for an integer k). It serves as the core cryptographic primitive in ECC, forming the basis for key generation and encryption. The security of ECC relies on the computational hardness of reversing this process, known as the Elliptic Curve Discrete Logarithm Problem (ECDLP).

Key Generation : The private key is a randomly chosen scalar k, and the corresponding public key is computed as $Q = kP$, where P is a publicly known base point.

Encryption and Decryption: ECC enables secure communication by allowing parties to encrypt and decrypt messages using combinations of their private keys and the recipient's public key.

F. Finite State Transducer

A Finite State Transducer is an abstract machine that reads input symbols and produces output symbols while transitioning through a finite number of states. It is a generalization of a Finite State Automaton and is commonly used for tasks such as lexical analysis, string transformation, and keyword indexing.

Formally, a finite state transducer T is a 6-tuple (Q, Σ, Γ, I, F, δ) such that :

- Q is a finite set, the set of states
- Σ is a finite set, called the input alphabet
- Γ is a finite set, called the output alphabet
- I is a subset of Q, the set of initial states
- F is a subset of Q, the set of final states
- $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \times Q$ (where ε is the empty string) is the transition relation. Each transition is a quadruple (q,a,b,q′), where:
    - $q \in Q$: current state

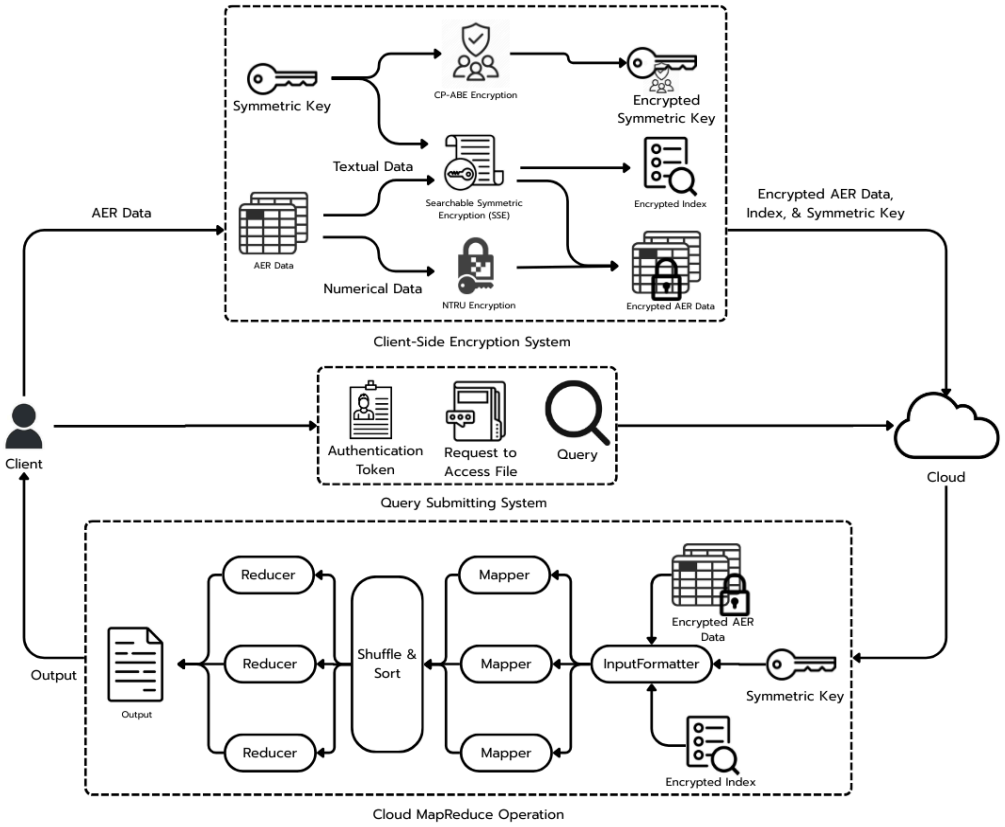


Fig 1. Proposed System Model

- a $\in \Sigma \cup \{\varepsilon\}$: input symbol or empty string (ε-transition)
- b $\in \Gamma \cup \{\varepsilon\}$: output symbol or empty string
- q′ $\in Q$: next state

The FSTs starts in any initial state $q_0 \in I$, processes an input string by following valid transitions, and produces an output string by concatenating all output symbols b encountered along the path. The computation is accepted if it ends in a final state $q_f \in F$.

IV. Our Proposed Scheme

This section presents the description of our proposed system model, the details of the system processes, as well as all formulas and algorithms pertaining to our system.

A. System Model Overview

Our proposed system model integrates secure encryption and cloud-based computation to ensure confidentiality and privacy in data processing of Adverse Event Reports (AER). It consists of two primary components: Client-Side Encryption System and Cloud MapReduce Operation. It also integrates CP-ABE encryption in order to secure data at-rest, allowing individuals to safely secure data at-rest on cloud, based on their own defined policies, and allowing only individuals with sufficient attributes to access and perform computations on the data.

B. File Preprocessing

The following section outlines the high-level overview for the stages of file preprocessing.

Stage 1 - System Setup: In the first stage, the trusted authority generates a CP-ABE master secret key and the public key, with curve parameters defined beforehand.

$$KeySetup(curveParameters) \rightarrow MSK,\ PK$$

The algorithm defines prime-order groups G_1, G_2 and a generator g. From this structure, the values α and β are randomly chosen from. The public key can then be expressed in the following form, where $e: G_1 \times G_2 \rightarrow GT$ is a bilinear pairing function.

$$PK = \{G_0, g, h = g^{\beta}, f = g^{\frac{1}{\beta}}, e(g,g)^{\alpha}\}$$

Additionally, the master secret key can be defined as follows.

$$MSK = \{\beta,\ g^{\alpha}\}$$

Stage 2 - Key Assignment: At the trusted authority, the key generation algorithm takes the master secret key (MSK), the public key (PK), and the set of attributes $S = \{att1, att2,...\} \in U$ associated with the individual who requests access to the system, then outputs a secret key (SK).

$$KeyGeneration(MSK, PK, S) \rightarrow SK$$

The CP-ABE secret key is then encrypted with a symmetric key K_{cloud} provided by a trusted cloud service, before being uploaded to the cloud for storage.

$$EncryptAES_{GCM}(SK, K_{cloud}, IV) = IV \,||\, CT_{SK}$$

Stage 3 - Client-Side Encryption: When an individual within the system wants to upload a file to the cloud storage, the **client-side system generates a persistent symmetric key** $SymKey$, which is stored and used in the encryption process as follows:

Textual Data Encryption: Columns defined as "textual" will undergo the process of searchable symmetric encryption inverted index construction. The algorithm separates and extracts keywords from individual cells, keeping track of the row appearance in a bitmap data structure.

$$keywordIndex\colon key \rightarrow b[i]$$

$$b[i] = \{1\ if\ keyword\ k\ appears\ in\ row\ i;\ 0\ otherwise\}$$

Once the extraction is complete, the keywords are hashed via HMAC with a symmetric key $SymKey$ to introduce deterministic obfuscation, allowing searchability without revealing the keywords.

$$HMAC_{SHA256}(key, SymKey) = h_{key}$$

Then, the associated bitmap is also encrypted with AES-GCM with a random IV.

$$EncryptAES_{GCM}(b[i], SymKey, IV) = IV \,||\, CT_{b[i]}$$

The hashed keyword and the encrypted row index data are organized into a finite search tree (FST), enabling efficient traversal and retrieval of row data. Lastly, the individual cells of the original file are encrypted with AES-GCM.

Numerical Data Encryption: Numerical data column undergoes encryption using the NTRU (Nth-degree Truncated Polynomial Ring Units) Encryption scheme. Each numerical data is encoded as a byte array before being encrypted with NTRU using the public key.

$$e = r \cdot h + m\ (mod\ q)$$

CP-ABE Key Wrapping: Lastly, the symmetric key $SymKey$ used in this process is encrypted with the CP-ABE key, previously assigned to the user. The user retrieves the same symmetric key K used to encrypt the CP-ABE secret key from a trusted cloud provider.

$$DecryptAES_{GCM}(IV \,||\, CT_{SK}, K_{cloud}) = SK$$

$$EncryptCPABE(SK,\ SymKey) = CT_{SymKey}$$

Transmission to Cloud Storage: The SSE index, encrypted data file, and CP-ABE encrypted symmetric key is transmitted to cloud storage for further processing.

C. Cloud MapReduce Operation

Once the encrypted data is stored in the cloud, computation is performed using a MapReduce-based framework to process encrypted queries securely. This framework consists of the following key components:

Client-Side Query Submitter: The client - through this component - provides the user authentication token, the file name to access, and the boolean query string to the trusted cloud proxy.

Before the query is sent, it is parsed for keyword tokens, which are then obfuscated with hashed versions of the keyword, using HMAC with the symmetric key stored on the client side.

$$Query = "k_1\ AND\ (k_2\ OR\ k_3)"$$

$$\downarrow$$

$$HMAC_{SHA256}(k_i, SymKey) = h_{ki}$$

$$\downarrow$$

$$ObfuscatedQuery = "h_{k1}\ AND\ (h_{k2}\ OR\ h_{k3})"$$

Trusted Cloud Proxy: This component then attempts to retrieve the specified file from the cloud, along with the associated CP-ABE-encrypted AES key, and the encrypted CP-ABE private key associated with the user who made the request. The proxy then decrypts the CP-ABE secret key with K_{cloud}.

$$DecryptAES_{GCM}(IV \,||\, CT_{SK}, K_{cloud}) = SK$$

Then, the proxy uses the CP-ABE secret key to attempt to decrypt the AES key. If decryption fails, the process ends with an error.

$$DecryptCPABE(SK,\ CT_{SymKey}) = SymKey$$

Otherwise, the proxy then proceeds to submit the job, along with the decrypted symmetric key, and the obfuscated query.

Input Formatting: For further optimization, a custom input format is used to parse through the obfuscated query before it reaches the mapper phase, ensuring that only relevant rows are split and fed into the mapper. The symmetric key passed into this point is used to decrypt the relevant row bitmaps obtained from the query parsing operation.

By constructing a binary tree-like structure from the boolean query, a recursive algorithm can be applied to perform set operations on the bitmap stored in the FST inverted index, allowing the correct number of rows to be retrieved.

$$ObfuscatedQuery = "h_{k1}\ AND\ (h_{k2}\ OR\ h_{k3})"$$

$$\downarrow$$

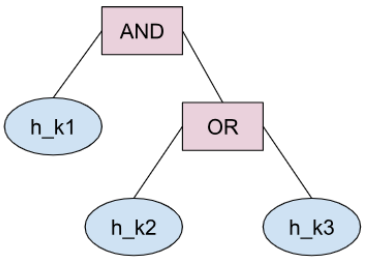


Fig 2. A tree representation of the sample obfuscated query string.

Algorithm 1: Boolean Query Evaluating

Input: queryTree, searchIndex

If node is a boolean node:
  Initialize result ← null
    Foreach clause **in** node:

```
        childResult ← EvaluateQuery(clause.subquery)
        If result == null:
            If clause == EXCLUDE:
                result ← ¬(UNIVERSAL_SET ∩ childResult)
            Else:
                result ← childResult
        Else:
            If clause == AND:
                result ← result ∩ childResult
            If clause == OR:
                result ← result ∪ childResult
            If clause == EXCLUDE:
                result ← result ∩ ¬childResult
    Return result
If node is a term node:
    Return searchIndex(term)
```

Map Phase: The mapper nodes receive encrypted row data from the designated file and process it according to the returned row indices from the query. Since the data is encrypted, the computations performed on the Mapper nodes follow encryption-preserving operations to maintain confidentiality.

Reduce Phase: The reducer nodes aggregate the results received from the Mappers and perform the final computations to generate encrypted outputs. Here, the homomorphically additive nature of NTRU encryption scheme is exploited in aggregating key-value pairs, ensuring that the data remains encrypted throughout the operation.

While this addition preserves correctness under encryption, it also causes the noise in the ciphertext to grow with each operation. To ensure that the accumulated noise does not exceed the decryption threshold, the system incorporates noise management strategies such as limiting the number of additions per reducer, and selecting an adequately large modulus q to accommodate bounded noise growth. This is done to ensure that encrypted results remain within the decryptable range without requiring expensive operations like bootstrapping.

V. Evaluation

In this section, we present the experiments related to the performance of client-side encryption operations and the accuracy and efficiency of secure MapReduce computation using encrypted data.

A. Comparison with Similar Implementations

EPiC [28] and SecureMR [29] both focus on enabling privacy-preserving MapReduce computations over encrypted data in untrusted cloud environments. EPiC uses somewhat homomorphic encryption combined with an arithmetization of Boolean patterns to achieve oblivious frequency counting with moderate overhead, while SecureMR analyzes and transforms MapReduce programs using partially homomorphic encryption schemes and static program analysis to minimize costly encryption scheme conversions during execution.

Our model aligns with these goals by enabling secure processing of sensitive Adverse Event Reports (AER) on MapReduce platforms, but it distinctly advances the state of the art by integrating post-quantum secure NTRU encryption for numerical data and searchable symmetric encryption (SSE) for textual fields to support efficient Sort-and-Shuffle operations. This hybrid approach ensures consistent ciphertexts for identical plaintexts, addressing a key limitation of NTRU's probabilistic encryption. Additionally, our use of CP-ABE enforces fine-grained access control, and the incorporation of indexing structures like Finite State Transducers (FSTs) enhances scalability and query efficiency. Unlike SecureMR, which incurs overhead due to client-server conversions to reconcile encryption schemes, our model minimizes such costly interactions by design, thereby improving practical performance in large-scale, privacy-sensitive cloud computations.

B. Client-Side Encryption & Decryption Operation

To evaluate the overhead introduced by encryption, we measured the time required for the client-side encryption process prior to uploading data to the MapReduce pipeline. This step involves parsing the FAERS CSV dataset, applying searchable symmetric encryption (SSE) to keyword-bearing textual fields, and encrypting numerical columns such as age using the NTRU encryption scheme. The encrypted dataset and the associated searchable index are then serialized and stored. This measurement provides insight into the preprocessing time required before the data becomes suitable for secure cloud-based computation. Each file size was tested 3 times, then the obtained time is averaged to minimize the variance.

Model / Feature	Homomorphic Encryption	Post-Quantum Security	Numerical Data Privacy	Categorical/ Text Privacy	Deterministic Encryption (Shuffle/Sort)	MapReduce Compatible	Access Control	Indexing Support	Trusted Client Dependency	Conversion Overhead
Our Model	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
EPiC	✓	✗	✓	✓	✗	✓	✗	✗	✗	✓
SecureMR	✓	✗	✓	✗	✗	✓	✗	✗	✓	✗

Fig 3. Summary of comparison on our model with EPiC and SecureMR.

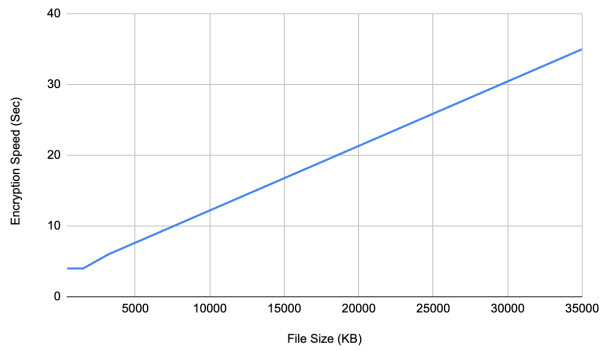


Fig 4. File encryption speed of our proposed scheme.

C. *Cloud MapReduce Operation*

To evaluate the correctness and efficiency of our privacy-preserving MapReduce pipeline, we used a dataset of adverse event reports from the FAERS repository. Our sample job executes a boolean keyword query on encrypted records and performs a summation of the encrypted patient age values from the matching entries.

As a baseline, we implemented a similarly traditional MapReduce job that operates on the plaintext version of the FAERS dataset, utilizing similar optimization strategies to our privacy-preserving approach: by constructing an unencrypted FST inverted index structure, and performing the job on the original unencrypted CSV files. This implementation serves as a reference point for measuring the correctness and performance overhead introduced by our privacy-preserving approach, while also allowing us to evaluate the impact of optimization techniques in both secure and non-secure contexts.

The MapReduce job was tested on a personal computer running Microsoft Windows 10 Pro, equipped with an Intel(R) Core(TM) i5-4200M CPU operating at 2.30 GHz and 8 GB of RAM. To simulate a scalable Hadoop environment, Docker and Docker Compose were used to deploy a pseudo-distributed cluster consisting of one NameNode and two DataNodes. The test job was implemented in Java 1.8.0 and submitted to the Hadoop framework using the standard MapReduce API. The input dataset, consisting of encrypted CSV data, was uploaded to the Hadoop Distributed File System (HDFS), and the job execution was verified through the Hadoop web UI and container logs. Performance metrics such as processing time and output correctness were evaluated using small to medium-sized chunks of encrypted data to emulate a cloud-like environment in a controlled local setting.

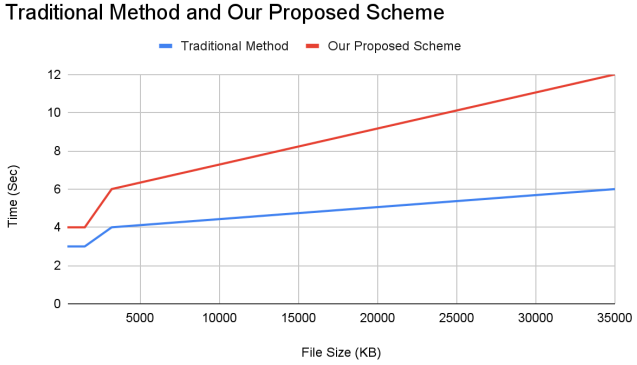


Fig 5. Results of MapReduce efficiency testing, comparing between the traditional method, and our proposed scheme.

D. *CP-ABE Encryption Overhead*

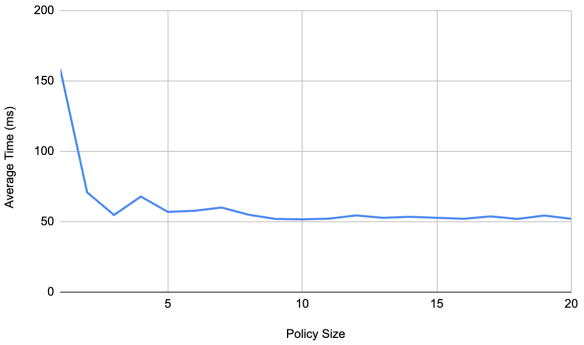


Fig 6. Results of CP-ABE encryption testing for average time per policy size.

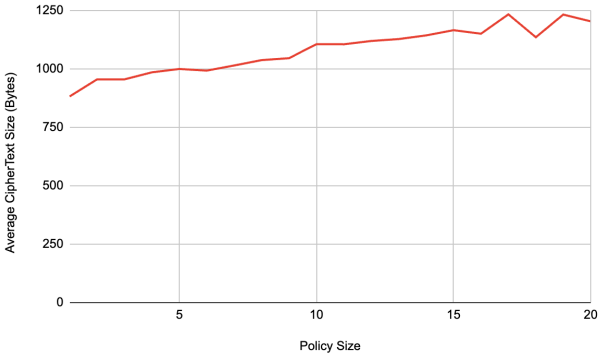


Fig 7. Results of CP-ABE encryption testing for average ciphertext size per policy size.

The experimental evaluation focused on measuring the encryption time overhead and ciphertext size growth in CP-ABE as a function of policy complexity. Policies were generated randomly with increasing numbers of attributes from 1 up to 20, combining attributes using logical AND and OR operators to simulate realistic and complex access structures.

The results, summarized in Figure 6 and 7, reveal two key insights: (1) Encryption time remained remarkably stable across increasing policy sizes, averaging between approximately 50 to 70 milliseconds for policies involving 2 to 20 attributes. An outlier was observed for policies of size 1, which incurred a higher average time of around 240 milliseconds—likely due to fixed overhead factors dominating in trivial cases. Overall, the data indicates that encryption computation in the tested CP-ABE scheme does not scale linearly with policy size, suggesting efficient handling or caching mechanisms within the underlying cryptographic operations. (2) Ciphertext size exhibited a roughly linear increase with policy size, growing from approximately 880 bytes for the smallest policies to about 1270 bytes for the largest tested policies. This aligns with theoretical expectations, as the ciphertext includes components proportional to the number of attributes embedded in the policy.

These findings suggest that, within the tested range, the CP-ABE scheme can handle complex policies without significant penalties in encryption time, though ciphertext size grows predictably with policy complexity. This behavior is beneficial for practical deployments where expressive access control policies are necessary but encryption latency must remain low.

VI. Results and Discussion

The experimental evaluation confirmed the correctness and practical feasibility of our privacy-preserving MapReduce framework when applied to encrypted clinical trial adverse event data. Our system combines NTRU-based additively homomorphic encryption for numerical values with searchable symmetric encryption (SSE) for keyword queries, enabling secure distributed computation without revealing sensitive information.

When compared to a baseline MapReduce job that operates on plaintext data with similar indexing optimizations but no cryptographic protections, our system exhibited an expected increase in query latency. Across datasets up to 100,000 rows, we observed a 2x to 2.5x increase in processing time, primarily due to the overhead introduced by cryptographic operations and secure index traversal. Despite this, the performance remained stable across increasing data sizes, suggesting that the system can scale well within the tested range.

Encryption performance was also measured and found to increase linearly with dataset size, at a manageable rate. For example, encrypting 10,000 rows took approximately 6 seconds, while 100,000 rows required around 35 seconds. This gradual growth indicates that our encryption pipeline, which applies cryptographic protection to all columns in the dataset, is efficient enough for practical use even on moderately sized datasets despite the computational intensity of encrypting every cell.

Although slower than its non-secure counterpart, our solution provides significantly stronger privacy guarantees, including protection against keyword inference and post-quantum threats. Optimizations such as pre-generated encrypted indices and selective retrieval of only relevant columns during the MapReduce operation helped mitigate overhead, contributing to reasonable processing times under a local pseudo-distributed Hadoop environment.

These results indicate that our framework, while incurring a measurable performance trade-off, delivers practical levels of efficiency alongside strong privacy preservation—making it a viable candidate for secure data analytics in regulated domains like clinical research.

VII. Conclusion

This paper presents a privacy-preserving MapReduce framework tailored for secure analytics over clinical adverse event reports. By integrating searchable symmetric encryption (SSE) with additively homomorphic, post-quantum encryption using NTRU, the system enables keyword-based filtering and secure numerical aggregation without revealing sensitive data to the processing nodes. The approach strikes a balance between performance and privacy, leveraging local optimization techniques and selective encryption to ensure practical scalability up to 100,000 records in a pseudo-distributed Hadoop environment.

Despite its advantages, the current implementation is limited in expressiveness due to the lack of fully homomorphic encryption, which constrains the system to basic aggregation tasks. Additionally, the static nature of the encrypted index may hinder flexibility in use cases involving frequent data updates or more complex query patterns. The reliance on per-session key management also presents challenges for long-term storage and collaborative workflows.

Future work will focus on extending the system's functionality through the incorporation of partially or fully homomorphic encryption techniques to support a broader range of computations. Enhancements to index structure, such as support for dynamic updates or hierarchical keyword queries, will also be explored. Finally, formal security proofs and load-aware scheduling strategies will be developed to improve both theoretical guarantees and practical responsiveness under real-world workloads.

References

[1] W. He, Y. Zhang and Y. Li, "Fast, Searchable, Symmetric Encryption Schema Supporting Ranked Search", *Symmetry*, vol. 14, no. 5, p. 1029, May 2022, doi : https://doi.org/10.3390/sym14051029

[2] I. Amorim and I. Costa, "Leveraging Searchable Encryption through Homomorphic Encryption: A Comprehensive Analysis", *Mathematics*, vol. 11, no. 13, p. 2948, July 2023, doi : https://doi.org/10.3390/math11132948

[3] D. Talapatra, A. Bag and S. Patranabis, "Multi-Client and Quantum-Resilient Searchable Symmetric Encryption", 2024

[4] H. Shinoki, H. Sato and M. Yoshino, "Fully Secure Searchable Encryption from PRFs, Pairings and Lattices", unpublished, November 2024

[5] Afsar Kamal, Khaleel Ahmad, Rosilah Hassan and Khujamatov Khalim, "NTRU Algorithm: Nth Degree Truncated Polynomial Ring Units", Springer International Publishing Cham, pp. 103-115, 2021.

[6] J. L. Raisaro et al., "MedCo: Enabling Secure and Privacy-Preserving Exploration of Distributed Clinical and Genomic Data," in IEEE/ACM Transactions on Computational Biology and Bioinformatics, vol. 16, no. 4, pp. 1328-1341, 1 July-Aug. 2019.

[7] Munjal K, Bhatia R. A systematic review of homomorphic encryption and its contributions in healthcare industry. Complex Intell Systems. 2022 May 3:1-28. doi: 10.1007/s40747-022-00756-z. Epub ahead of print. PMID: 35531323; PMCID: PMC9062639.

[8] "MedCo - Enabling the Secure and Privacy-Preserving Exploration of Distributed Clinical and *Omics Cohorts in the Swiss Personalized Health Network". SPHN. Retrieved 2021-04-19.

[9] S. Xie, F. Wu, X. Zhang, W. Yao and Z. Zheng, "Similarity Test for Privacy-Preserving Medical Data Sharing Based on NTRU Encryption," 2019 IEEE 9th International Conference on Electronics Information and Emergency Communication (ICEIEC), Beijing, China, 2019, pp. 1-4, doi: 10.1109/ICEIEC.2019.8784488.

[10] Lei Xu, Chungen Xu, Joseph K. Liu, Cong Zuo, Peng Zhang, Building a dynamic searchable encrypted medical database for multi-client, Information Sciences, Volume 527, 2020, Pages 394-405, ISSN 0020-0255.

[11] Kexin Xu, Benjamin Hong Meng Tan, Li-Ping Wang, Khin Mi Mi Aung, Huaxiong Wang, Threshold Homomorphic Encryption From Provably Secure NTRU, The Computer Journal, Volume 66, Issue 12, December 2023, Pages 2861–2873.

[12] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, "Searchable symmetric encryption: improved definitions and efficient constructions," *IACR Cryptology ePrint Archive*, vol. 2006, no. 210, pp. 1–33, 2011. [Online]. Available: https://eprint.iacr.org/2006/210

[13] K. Men, H. Li, B. Wang and M. Liu, "An NTRU-based Homomorphic Encrypted Data Analysis System", 2023 IEEE 3rd International Conference on Information Technology, May 2023, doi: 10.1109/ICIBA56860.2023.10164945

[14] D. Kong, J. Li, and H. Tian, "Easy-ABE: An Easy Ciphertext-Policy Attribute-Based Encryption," *IACR Cryptol. ePrint Arch.*, vol. 2023, no. 1814, pp. 1–35, 2023. [Online]. Available: https://eprint.iacr.org/2023/1814

[15] L. Bai, L. Bai, Y. Li, and Z. Li, "Research on Noise Management Technology for Fully Homomorphic Encryption," *IEEE Access*, vol. 12, pp. 135564–135576, Sep. 2024, doi: 10.1109/ACCESS.2024.3461729.

[16] A. Bag, D. Talapatra, A. Rastogi, S. Patranabis, and D. Mukhopadhyay, "TWo-IN-one-SSE: Fast, Scalable and Storage-Efficient Searchable Symmetric Encryption for Conjunctive and Disjunctive Boolean Queries," *Proceedings on Privacy Enhancing Technologies*, vol. 2023, no. 1, pp. 115–139, 2023, doi: 10.56553/popets-2023-0008.

[17] F. Liu, K. Xue, J. Yang, J. Zhang, Z. Huang, and J. Li, "Volume-Hiding Range Searchable Symmetric Encryption for Large-Scale Datasets," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 3597–3609, Jul.–Aug. 2024, doi: 10.1109/TDSC.2023.3335304.

[18] C. Bonte, I. Shenko, J. Park, H. V. L. Pereira, and N. P. Smart, "FINAL: Faster FHE instantiated with NTRU and LWE," in Advances in Cryptology – ASIACRYPT 2022, Lecture Notes in Computer Science, vol. 13792, Y. Dodis and T. Shrimpton, Eds. Cham, Switzerland: Springer, Jan. 2023, pp. 173–202. doi: 10.1007/978-3-031-22966-4_7.

[19] J. Bi and L. Han, "Lattice Attacks on NTRU Revisited," IEEE Access, vol. 9, pp. 66218–66222, Apr. 2021, doi: 10.1109/ACCESS.2021.3076598.

[20] A. Kharangate, "Asynchronous Merkle Trees," arXiv preprint arXiv:2311.17441 [cs.DC], Nov. 2023. doi: 10.48550/arXiv.2311.17441.

[21] M. T. Marinov, "A Bloom Filter Application for Processing Big Datasets through MapReduce Framework," in Proceedings of the 2021 International Conference on Information Technologies (InfoTech), St. St. Constantine and Elena, Bulgaria, Sep. 2021, pp. 1-5. doi: 10.1109/InfoTech52438.2021.9548638.

[22] S. Saha and D. V. N. Siva Kumar, "Enhancement of Precision on Cloud Data using Multi-Layered Bloom Filter," in *Proceedings of the 2021 IEEE 18th India Council International Conference (INDICON)*, Guwahati, India, Dec. 2021, pp. 1–8. doi: 10.1109/INDICON52576.2021.9691668.

[23] J. VenkataGiri and A. S. R. Murty, "Elliptical Curve Cryptography Design Principles," in *Proc. 2021 Int. Conf. Recent Trends Electronics, Inf., Commun. & Technol. (RTEICT)*, Bengaluru, India, Aug. 2021, pp. 1–5. doi: 10.1109/RTEICT52294.2021.9573662.

[24] Md. N. S. Shishir and M. A. Yousuf, "Performance Enhancement of Hadoop MapReduce by Combining Data Inside the Mapper," in *Proceedings of the 2nd International Conference on Robotics,*

[25] *Electrical and Signal Processing Techniques (ICREST)*, Dhaka, Bangladesh, Jan. 2021, pp. 239–243. doi: 10.1109/ICREST51555.2021.9331097.

[26] H. B. Abdalla, Y. Kumar, Y. Zhao, and D. Tosi, "A Comprehensive Survey of MapReduce Models for Processing Big Data," Big Data and Cognitive Computing, vol. 9, no. 4, p. 1-37, Mar. 2025, doi: 10.3390/bdcc9040077.

[27] A. K. Ka, "Easy-ABE: An easy ciphertext-policy attribute-based encryption," *IACR Cryptol. ePrint Arch.*, vol. 2023, no. 1814, pp. 1–35, 2023. [Online]. Available: https://eprint.iacr.org/2023/1814.pdf

[28] Blass, E.-O., Noubir, G., & Vo-Huu, T. D. (2012). EPiC: Efficient Privacy-Preserving Counting for MapReduce. Cryptology ePrint Archive, Report 2012/452. https://eprint.iacr.org/2012/452

[29] Dong, Y., Milanova, A., & Dolby, J. (2018, February). SecureMR: Secure MapReduce Using Homomorphic Encryption and Program Partitioning [Poster]. Presented at the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '18), Vienna, Austria. https://doi.org/10.1145/3178487.3178520